

Wednesday — Week 6

HyDE + Query Transformation + Query Decomposition

HyDE · Step-Back Prompting · Query Decomposition · Multi-Query · Query Routing

By the end of today you will be able to:

Explain why the original user query is often a poor retrieval signal and how to improve it

Implement HyDE (Hypothetical Document Embeddings) and explain why it improves recall

Apply step-back prompting to broaden retrieval for narrow or overly specific queries

Decompose complex multi-part questions into sub-queries and merge the retrieved contexts

Build a multi-query retriever that generates query variations and deduplicates results

Select the right query transformation technique for a given query failure mode

NOTE

Tuesday recap: two-stage retrieval (retrieve-20, re-rank to top-3). Bi-encoder vs cross-encoder. bge-reranker-base implementation. Cohere Rerank API. recall@5 measurement. Today: the third layer of RAG quality improvement — transforming the query itself before retrieval.

June 10, 2026

Online (Zoom) · 10:00 AM – 1:00 PM · Mon–Thu | In-Person Friday 10:00 AM – 1:00 PM

Crewlogix — The Institute of Technology · Gulberg III, Lahore

crewlogix

The Institute of Technology

Session — Wednesday June 10

3 hours

■ Online — Zoom · 10:00 AM – 1:00 PM

The Query Problem (15 min)

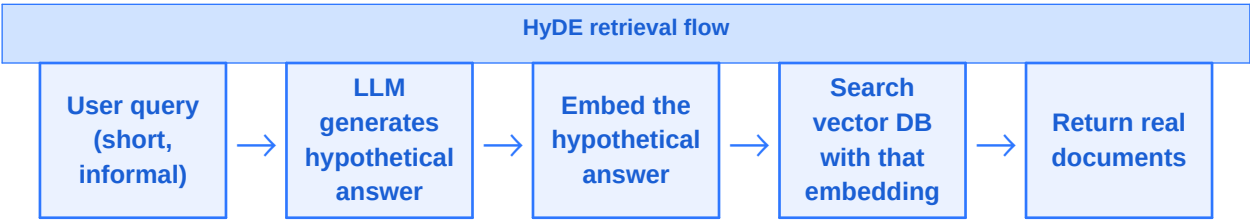
We have upgraded retrieval (hybrid search) and ranking (re-ranker). But there is a third weak point: the query itself. Users often write queries that are poor retrieval signals — too short, too specific, incorrectly phrased, or requiring multi-step reasoning.

Query problem type	Example	What goes wrong
Too short / vague	"company registration"	Matches hundreds of chunks — precision is poor
Jargon mismatch	"what's the hassle for registering my biz"	Embedding of informal language may not match formal regulatory text
Asking for something not in documents	"how does SECP compare to SEC in the US?"	Documents only cover SECP — comparison context is absent
Multi-part question	"What are the fees, timeline and documents needed for SECP registration?"	Three separate sub-topics — one retrieval pass likely misses at least one
Too specific / narrow	"Form 29 subsection 3 paragraph 2 bullet point b"	Hyper-specific query matches nothing — the document uses different wording

Topic 1 — HyDE: Hypothetical Document Embeddings (45 min)

HyDE (Gao et al., 2022) solves the jargon mismatch problem elegantly. Instead of embedding the user's query directly, it asks the LLM to generate a *hypothetical answer* to the query — as if the perfect document existed. That hypothetical answer is then embedded and used for retrieval. The hypothesis lives in "document space" rather than "query space" and matches real documents far better.

1.1 How HyDE works



The intuition: A user asks "what's the hassle for registering a company?" The embedding of this informal query is distant from formal regulatory text. But if the LLM generates "To register a company in Pakistan, you must file Form I with SECP, provide a memorandum of association, and pay a registration fee..." — that hypothetical answer shares the vocabulary and style of the actual regulatory documents and retrieves them accurately.

1.2 HyDE implementation

```
# HyDE: generate a hypothetical answer, then embed it for retrieval
from openai import OpenAI
from sentence_transformers import SentenceTransformer

llm = OpenAI(api_key="INSTITUTE_KEY")
emb_model = SentenceTransformer("all-MiniLM-L6-v2")

def hyde_retrieve(query, collection, top_k=5):
    """HyDE: generate hypothetical answer, embed it, retrieve."""

    # Step 1: generate a hypothetical answer
    hypo_prompt = f"""Write a short, factual answer (2-3 sentences)
    to the following question, as if you were a Pakistani regulatory expert.
    Do not say you don't know – generate a plausible answer.

    Question: {query}

    Answer: """

    hypo_answer = llm.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": hypo_prompt}],
        max_tokens=150,
        temperature=0.0
    ).choices[0].message.content

    # Step 2: embed the hypothetical answer (not the query)
    hypo_emb = emb_model.encode([hypo_answer]).tolist()

    # Step 3: retrieve using the hypothetical embedding
    results = collection.query(
        query_embeddings=hypo_emb,
        n_results=top_k,
        include=["documents", "distances"]
    )
    return results["documents"][0]
```

1.3 HyDE vs standard retrieval — side by side

Query	Standard retrieval top-1	HyDE top-1
"what's the hassle for registering my biz"	Generic document about "business registration hassle" — low precision	"To register a company in Pakistan, file Form I with SECP..." — correct, formal document
"how do I avoid paying too much tax"	Documents about tax avoidance strategies (possibly wrong ones)	"Taxpayers may claim deductions under Section 60B for eligible investments..." — correct FBR document
"SECP late fees"	Both methods work fine — short exact query	Both methods work fine — no meaningful difference for exact queries

NOTE

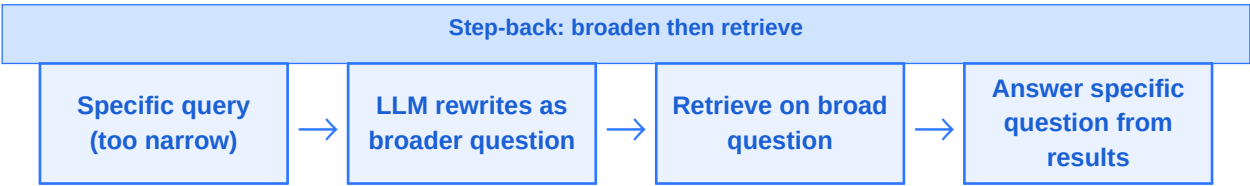
HyDE adds one LLM call per query — increases latency by ~300–500ms. Only use it when standard retrieval recall is poor on your query set. Measure before adding it: if recall@5 is already above 0.80, HyDE may not justify the extra cost.

EXAM TIP

AWS exam: "A RAG system retrieves poor results when users ask informal questions about formal regulatory documents. Which technique addresses the vocabulary mismatch between informal queries and formal document language?" Answer: HyDE — generates a hypothetical answer in the document's formal vocabulary, then embeds that for retrieval.

Topic 2 — Step-Back Prompting (25 min)

Step-back prompting (Zheng et al., 2023) handles the opposite problem from HyDE: queries that are *too specific* and fail because the exact narrow phrasing does not match any document. Step-back asks the LLM to rewrite the query as a broader, more general version before retrieval.



Original specific query	Step-back broadened query
"Form 29 subsection 3 paragraph 2 requirements"	"What are the requirements for filing Form 29 with SECP?"
"PKR 47,500 penalty for 3-month late annual return"	"What are the penalties for late annual return filing with SECP?"
"EOBI contribution rate for construction sector workers in Punjab 2024"	"What are the EOBI contribution rates for workers in Pakistan?"

```
# Step-back implementation
def step_back_retrieve(query, collection, emb_model, llm, top_k=5):
    """Broaden the query before retrieval."""
```

```
step_back_prompt = f"""Rewrite the following specific question
as a broader, more general question that covers the same topic.
Return ONLY the rewritten question, nothing else.
```

```
Original: {query}
Broader version: """
```

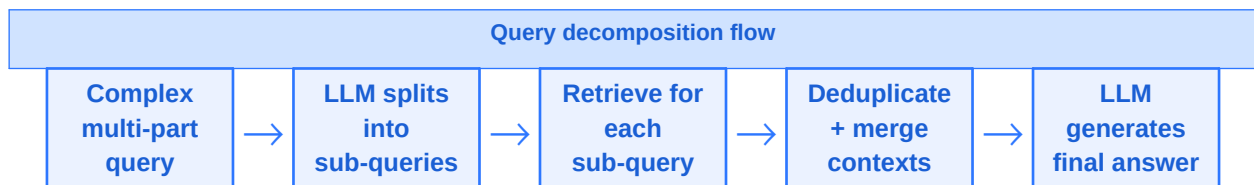
```
broad_query = llm.chat.completions.create(
    model = "gpt-3.5-turbo",
    messages = [{"role": "user", "content": step_back_prompt}],
    max_tokens = 60,
    temperature = 0.0
).choices[0].message.content.strip()
```

```
print(f"Step-back: '{query}' → '{broad_query}'")
```

```
broad_emb = emb_model.encode([broad_query]).tolist()
results = collection.query(
    query_embeddings = broad_emb,
    n_results = top_k,
    include = ["documents", "distances"]
)
return results["documents"][0]
```

Topic 3 — Query Decomposition (35 min)

Multi-part questions contain several independent sub-questions. A single retrieval pass cannot retrieve the best context for all parts simultaneously. Query decomposition splits the question into sub-queries, retrieves separately for each, then merges the contexts before generating.



```
# Query decomposition implementation
import json

def decompose_and_retrieve(query, collection, emb_model, llm, top_k=3):
    """Split multi-part query, retrieve per part, merge contexts."""

    decompose_prompt = f"""Split the following question into
2-4 simpler sub-questions that can be answered independently.
Return a JSON array of strings. Example: ["q1", "q2", "q3"]

Question: {query}
```

```

Sub-questions (JSON array only):"""

response = llm.chat.completions.create(
    model = "gpt-3.5-turbo",
    messages = [{"role": "user", "content": decompose_prompt}],
    max_tokens = 150,
    temperature = 0.0
).choices[0].message.content

sub_queries = json.loads(response)
print(f"Sub-queries: {sub_queries}")

# Retrieve for each sub-query, collect all chunks
all_chunks = []
for sub_q in sub_queries:
    emb = emb_model.encode([sub_q]).tolist()
    results = collection.query(
        query_embeddings = emb,
        n_results = top_k,
        include = ["documents"]
    )
    all_chunks.extend(results["documents"][0])

# Deduplicate while preserving order
seen, unique = set(), []
for chunk in all_chunks:
    if chunk not in seen:
        seen.add(chunk)
        unique.append(chunk)

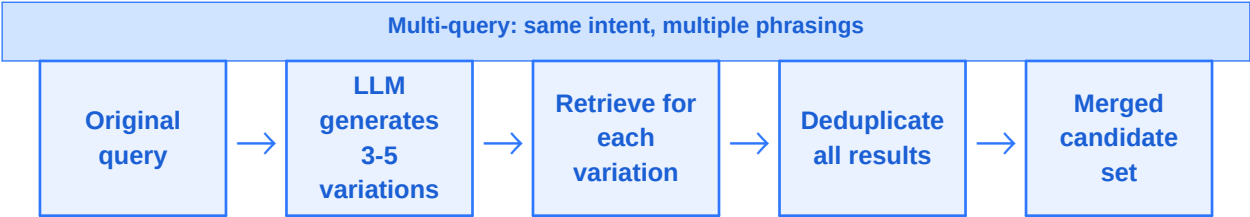
return unique

# Test with a complex Pakistani regulatory query
query = "What are the fees, timeline, and required documents for SECP company registration?"
chunks = decompose_and_retrieve(query, collection, emb_model, llm)
# Sub-queries: [
# "What are the fees for SECP company registration?",
# "How long does SECP company registration take?",
# "What documents are required for SECP company registration?"
# ]

```

Topic 4 — Multi-Query Retriever (30 min)

Multi-query retrieval generates multiple *variations* of the same query from different angles, retrieves for each variation, and deduplicates the results. This improves recall by catching documents that match one phrasing but not another.



```
# LangChain MultiQueryRetriever – built-in implementation
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain_openai import ChatOpenAI

llm_langchain = ChatOpenAI(
    model = "gpt-3.5-turbo",
    temperature = 0,
    api_key = "INSTITUTE_KEY"
)

multi_query_retriever = MultiQueryRetriever.from_llm(
    retriever = vector_retriever, # base retriever from Monday
    llm = llm_langchain
)

# Automatically generates variations and retrieves for all
query = "SECP company registration requirements"
results = multi_query_retriever.get_relevant_documents(query)
print(f"Retrieved {len(results)} unique chunks (deduplicated)")
```

Under the hood, MultiQueryRetriever generates 3–5 variations like: "How to register a company with SECP Pakistan?", "SECP registration process and requirements", "Documents needed for SECP company formation" — retrieves for each, deduplicates, and returns the union.

Topic 5 — Choosing the Right Query Transformation (15 min)

Failure mode	Symptom	Best technique	Extra LLM calls
Vocabulary mismatch	Informal query misses formal documents	HyDE — generates formal hypothetical answer	1 call
Query too specific / narrow	Hyper-specific query returns 0 results	Step-back — broadens the query	1 call

Multi-part question	Complex query; some sub-topics not covered in answer	Query decomposition — splits into sub-queries	1 call + 1 per sub-query
Ambiguous phrasing	Same query sometimes works, sometimes fails	Multi-query — generates variations	1 call
Good queries, poor ranking	Right topic retrieved but wrong chunk is #1	Re-ranking (Tuesday) — do not transform, re-rank	0 extra LLM calls
All working well	recall@5 already above 0.80	Do nothing — each technique adds latency	0

Exercise — Diagnose and apply query transformation (25 min)

1. Take your Pakistani document collection. Write 4 queries: one informal, one too specific, one multi-part, one ambiguous.
2. Run each through standard retrieval. For each, identify which failure mode it hits using the table above.
3. Apply the corresponding technique (HyDE, step-back, decomposition, or multi-query).
4. Compare recall@5 before and after for each query. Which technique gave the biggest improvement?

Day Summary — Wednesday Week 6

Technique	Problem it solves	Latency cost	When to use
HyDE	Informal/jargon query ↔ formal document vocabulary gap	+1 LLM call (~400ms)	recall < 0.70 on informal queries
Step-back	Hyper-specific query returns no results	+1 LLM call (~200ms)	Narrow queries returning 0 results
Decomposition	Multi-part question — single pass misses sub-topics	+1 call per sub-query	Questions with 3+ distinct parts
Multi-query	Ambiguous phrasing — sometimes hits, sometimes misses	+1 LLM call	Inconsistent retrieval on same topic

DO THIS

Tonight: AWS Coursera Week 4 content. Thursday we implement the full RAGAS evaluation framework — measuring all 4 metrics on your complete RAG pipeline. Bring your pipeline from this week.

